

Django: Multiple Apps with Views & URLs

Quick Revision Notes • Blog & Shop Modular Architecture Guide

Video #8 (Hindi)

Channel: Mohit Decodes

Concept: Modular URL Routing & include()

1. Key Concept: Modular Architecture क्यों ज़रूरी है?

Django में जब प्रोजेक्ट बड़ा होता है, तो सारा कोड एक ही जगह नहीं रखा जाता। अलग-अलग फीचर्स (जैसे Blog, E-commerce Shop, User Auth) के लिए अलग-अलग **Apps** बनाई जाती हैं।

- **Separation of Concerns:** हर ऐप का अपना `views.py` और अपना `urls.py` होता है।
- **Reusability & Maintenance:** ऐप को किसी अन्य प्रोजेक्ट में भी आसानी से प्लग/अनप्लग किया जा सकता है।
- **Project `urls.py`:** मेन प्रोजेक्ट का URL फ़ाइल सिर्फ `include()` के ज़रिए ऐप के URLs को डायरेक्ट (route) करता है।

2. Complete Step-by-Step Implementation Workflow

Step 1: Project & Apps Create करना

टर्मिनल / CMD में नया प्रोजेक्ट और दो अलग-अलग ऐप्स (blog और shop) बनाएं:

```
# 1. Django Project बनाना
django-admin startproject myproject1

# 2. प्रोजेक्ट डायरेक्टरी में जाना
cd myproject1

# 3. Blog और Shop नाम की दो Apps बनाना
python manage.py startapp blog
python manage.py startapp shop
```

Step 2: INSTALLED_APPS में Apps Register करना

फ़ाइल: `myproject1/settings.py`

Best Practice Note: केवल View/URL चलाने के लिए रजिस्ट्रेशन मिस होने पर कभी एरर नहीं आता, लेकिन Models, Migrations और Templates के लिए Apps को `INSTALLED_APPS` में जोड़ना अनिवार्य होता है।

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    # Custom Apps registered here:
    'blog',
    'shop',
]
```

Step 3: Apps के अंदर Views डिफाइन करना (HTTP Response)

`blog/views.py`

```
from django.http import HttpResponse

def home(request):
    return HttpResponse("Blog Home Page")
```

`shop/views.py`

```
from django.http import HttpResponse

def home(request):
    return HttpResponse("Shop Home Page")
```

```
def about(request):  
    return HttpResponse("Blog About Page")
```

```
def product(request):  
    return HttpResponse("Shop Products Page")
```

Step 4: App-level urls.py बनाना और Routes डिफाइन करना

डिफॉल्ट रूप से App फोल्डर में urls.py नहीं होता। हमें दोनों ऐप्स में urls.py खुद क्रिएट करना होता है:

blog/urls.py

```
from django.urls import path  
from . import views  
  
urlpatterns = [  
    path('', views.home, name='blog_home'),  
    path('about/', views.about, name='blog_about'),  
]
```

shop/urls.py

```
from django.urls import path  
from . import views  
  
urlpatterns = [  
    path('', views.home, name='shop_home'),  
    path('products/', views.product, name='shop_products'),  
]
```

Step 5: Main Project में App URLs को include() करना

फ़ाइल: myproject1/urls.py (प्रोजेक्ट का मुख्य URL कॉन्फिगरेशन)

```
from django.contrib import admin  
from django.urls import path, include  
  
urlpatterns = [  
    path('admin/', admin.site.urls),  
    path('blog/', include('blog.urls')), # Blog app routes include  
    path('shop/', include('shop.urls')), # Shop app routes include  
]
```

Step 6: Development Server Run करना

```
python manage.py runserver
```

3. URL Routing & Output Mapping Summary

Browser URL Endpoint	Matched App	Target View Function	Output / Response Displayed
http://127.0.0.1:8000/blog/	blog	views.home	Blog Home Page
http://127.0.0.1:8000/blog/about/	blog	views.about	Blog About Page
http://127.0.0.1:8000/shop/	shop	views.home	Shop Home Page
http://127.0.0.1:8000/shop/products/	shop	views.product	Shop Products Page
http://127.0.0.1:8000/admin/	Django Core	Admin Panel	Django Admin Login Page

4. Quick Revision Cheat Sheet / Key Takeaways

- **include('app_name.urls')**: यह Django का बिल्ट-इन फंक्शन है जो प्रोजेक्ट-लेवल URLs को ऐप-लेवल URLs से जोड़ता है।
- **Relative Import (from . import views)**: करंट डायरेक्टरी से व्यूज को एक्सेस करने के लिए डॉट (.) नोटेशन का उपयोग होता है।
- **Empty Path ''**: जब मेन रूट पर ही सब-होमपेज दिखाना हो (जैसे /blog/), तब ऐप के URL पैटर्न में पाथ स्ट्रिंग को खाली '' रखा जाता है।
- **Trailed Slash /**: Django बाय-डिफॉल्ट APPEND_SLASH = True फॉलो करता है, इसलिए 'about/' या 'products/' लिखना मानक तरीका है।